

Joanna Sierra-Mendoza

Professor Ismaily

DSC 650: Big Data

28 February 2026

Distributed Data Pipeline for Water Potability Analysis

Introduction:

Modern data systems often require multiple tools working together to move, process, and store large datasets efficiently. The primary goal of this project was to implement a distributed data pipeline using components of the Hadoop ecosystem and to understand how data flows through different stages of a big data architecture. The pipeline was built and executed within a Google Cloud virtual machine environment using PuTTY for remote access and management.

The pipeline shows how data can be ingested, stored, processed, and analyzed using several integrated technologies. Apache NiFi was used for ingestion, HDFS for distributed storage, Hive for structured querying, Spark MLlib for machine learning analysis, and HBase for storing model evaluation results. Together, these components form a complete workflow that shows how modern distributed data pipelines operate in real-world big data environments.

Dataset Source:

<https://www.kaggle.com/datasets/adityakadiwal/water-potability/data>

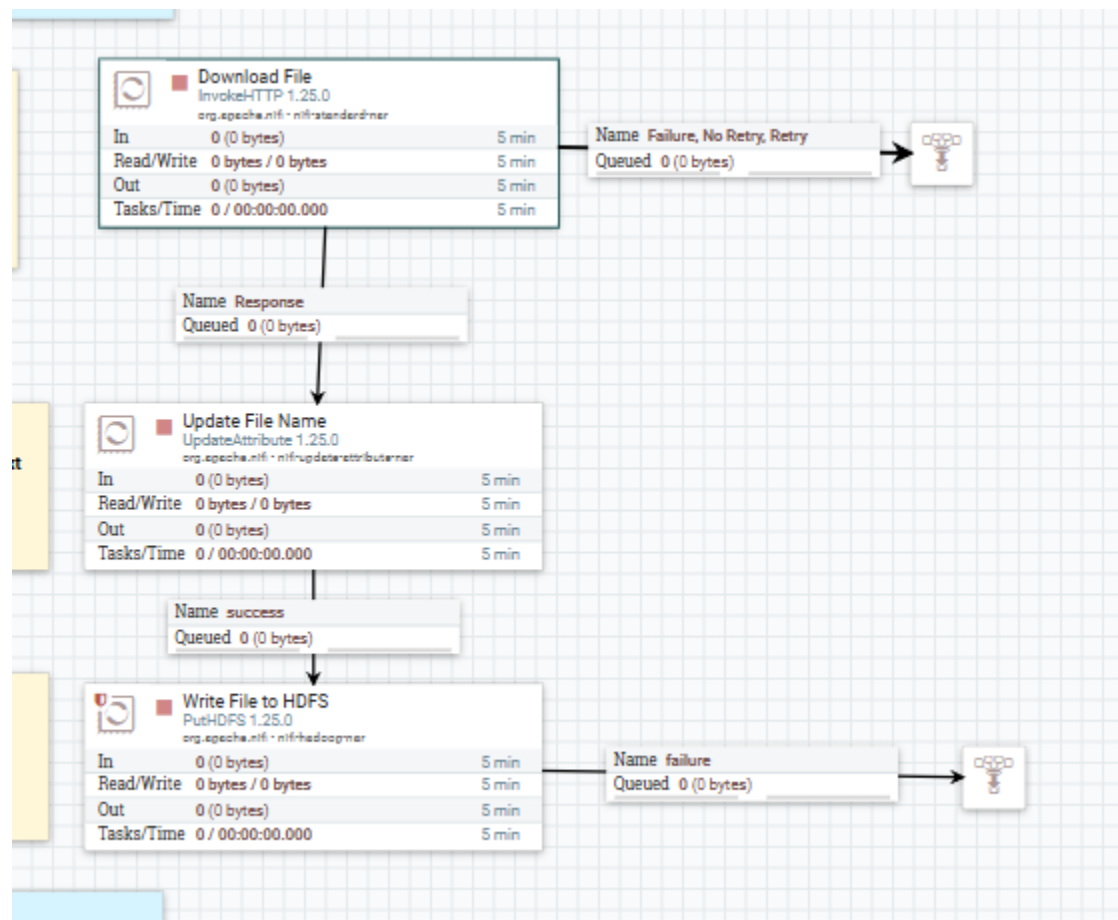
<https://raw.githubusercontent.com/josierra21/bigData/refs/heads/main/water.csv>

Pipeline Overview:

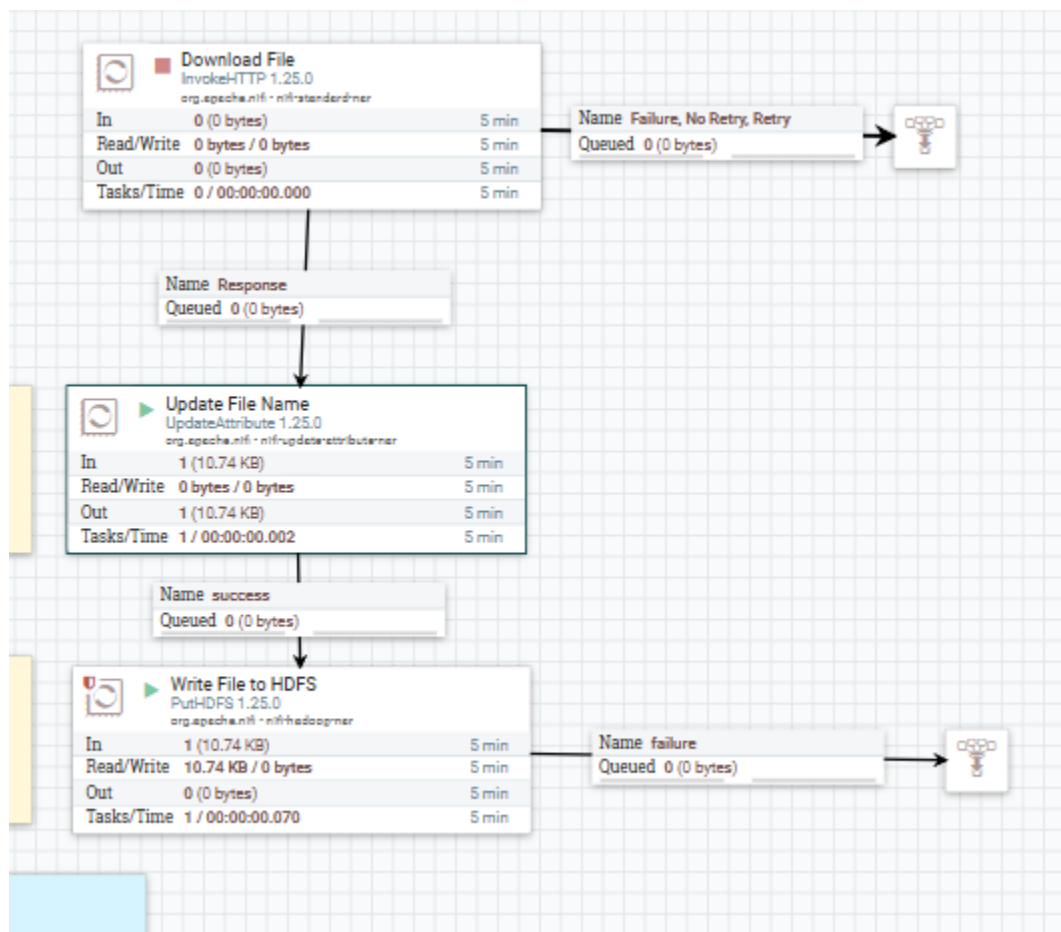
GitHub Dataset (water.csv) → **Apache NiFi** (Data Ingestion) → **HDFS** (Distributed Storage) → **Hive** (Structured Query & Aggregation) → **Apache Spark** (Machine Learning Model Training) → **HBase** (Storage of Model Evaluation Metrics)

Objective 1: NiFi Data Ingestion into HDFS

NiFi flow design:



Flow running with data:



Dataset successfully ingested into HDFS:

```
bash-5.0# hdfs dfs -ls /tmp
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/usr/program/hadoop/share/hadoop/common/lib/slf4j-log4j12-1.7.25.jar!/org/slf4j/impl/Static
LoggerBinder.class]
SLF4J: Found binding in [jar:file:/usr/program/tez/lib/slf4j-log4j12-1.7.10.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/usr/program/hive/lib/log4j-slf4j-impl-2.10.0.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.slf4j.impl.Log4jLoggerFactory]
2026-03-02 10:48:09,990 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-j
ava classes where applicable
Found 2 items
drwx-wx-wx - root supergroup 0 2026-03-02 10:36 /tmp/hive
-rw-r--r-- 3 joanna supergroup 10993 2026-03-02 10:46 /tmp/water.csv
```

InvokeHTTP (Download File): Pulled the water.csv dataset from the GitHub raw URL and created a flowfile containing the CSV contents.

UpdatedAttributes (Update File Name): Set the filename attribute to water.csv so the output file has a consistent name when written to storage.

PutHDFS (Write File to HDFS): Wrote the flowfile into HDFS under the configured directory /tmp. Successful ingestion was verified using “hdfs dfs -ls /tmp”, which showed water.csv.

Objective 2: Hive Managed Table

Data successfully loaded into hive:

```
hive> LOAD DATA INPATH '/tmp/water.csv'
> INTO TABLE water_quality;
Loading data to table default.water_quality
OK
Time taken: 2.744 seconds
hive> █
```

Query: Count records where water is acidic

```
hive> SELECT COUNT(*) AS acidic_water
> FROM water_quality
> WHERE ph < 7;
2026-03-03 00:15:31,217 INFO [9b5ab5e0-2752-43ce-b7c1-dddc79dd6ddb main] reducesink.VectorReduceSinkEmpty
g.apache.hadoop.hive.q1.plan.VectorReduceSinkInfo@51c4992e
Query ID = root_20260303001530_5fed07ca-8108-4ce8-9f43-f3b7cc60087e
Total jobs = 1
Launching Job 1 out of 1
Status: Running (Executing on YARN cluster with App id application_1772496128396_0004)

-----
VERTICES      MODE      STATUS  TOTAL  COMPLETED  RUNNING  PENDING  FAILED  KILLED
-----
Map 1 ..... container  SUCCEEDED    1          1          0          0          0          0
Reducer 2 ..... container  SUCCEEDED    1          1          0          0          0          0
-----
VERTICES: 02/02 [=====>>>] 100% ELAPSED TIME: 5.41 s
-----
OK
46
Time taken: 6.453 seconds, Fetched: 1 row(s)
█
```

The schema was designed to closely match the structure of the CSV file. All water quality measurement columns such as pH, Hardness, Solids, and other chemical properties were defined as DOUBLE because they contain decimal values and require numerical precision for accurate analysis. The Potability column was defined as INT since it represents a binary classification value of 0 or 1. The table was stored as a TEXTFILE with comma delimiters to match the format of the source CSV file. This schema ensures accurate data typing while allowing Hive to correctly interpret the raw data during query execution.

Objective 3: Environment Setup

Successful installation of HappyBase and Numpy packages:

```
bash-5.0# pip3 install numpy happybase
Collecting numpy
  Downloading numpy-1.21.6.zip (10.3 MB)
    |#####| 10.3 MB 5.5 MB/s
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing wheel metadata ... done
Collecting happybase
  Downloading happybase-1.3.0-py2.py3-none-any.whl (26 kB)
Collecting importlib-resources
  Downloading importlib_resources-5.12.0-py3-none-any.whl (36 kB)
Collecting six
  Downloading six-1.17.0-py2.py3-none-any.whl (11 kB)
Collecting thriftpy2>=0.4
  Downloading thriftpy2-0.6.0.tar.gz (996 kB)
    |#####| 996 kB 39.3 MB/s
  Installing build dependencies ... done
  WARNING: Missing build requirements in pyproject.toml for thriftpy2>=0.4 from https://files.pythonhosted.org/packages/cc/7c/bc038fb868b6b5474101197d42866fbc61839c4f739816587a7
  thriftpy2-0.6.0.tar.gz#sha256=621c263f99274d51a9aldeecf301845f1408d497bdafed682db6155132a99cf4 (from happybase).
  WARNING: The project does not specify a build backend, and pip cannot fall back to setuptools without 'wheel'.
  Getting requirements to build wheel ... done
  Installing backend dependencies ... done
  Preparing wheel metadata ... done
Collecting zipp>=3.1.0; python_version < "3.10"
  Downloading zipp-3.15.0-py3-none-any.whl (6.8 kB)
Collecting typing-extensions>=3.7.4; python_version < "3.8"
  Downloading typing_extensions-4.7.1-py3-none-any.whl (33 kB)
Collecting ply<4.0,>=3.4
  Downloading ply-3.11-py2.py3-none-any.whl (49 kB)
    |#####| 49 kB 6.0 MB/s
Building wheels for collected packages: numpy, thriftpy2
  Building wheel for numpy (PEP 517) ... done
  Created wheel for numpy: filename=numpy-1.21.6-cp37-cp37m-linux_x86_64.whl size=16644162 sha256=27663bf34cab7ad5ed679a840ccf9b9db92f41420fae25942e7552b0b05dee04c
  Stored in directory: /root/.cache/pip/wheels/4e/7e/9e/0fde042ccff2493994076dac9c3fbd78feb44c3bd94eb386a
  Building wheel for thriftpy2 (PEP 517) ... done
  Created wheel for thriftpy2: filename=thriftpy2-0.6.0-cp37-cp37m-linux_x86_64.whl size=1479828 sha256=362120927bcc5cbce57d31234f77cbb112f7c3f7a4250c5a4cc82935ead12bd0
  Stored in directory: /root/.cache/pip/wheels/ce/e1/d3/38770f98821c966d3ad35b392293b6e0bc189e89580338db63
Successfully built numpy thriftpy2
Installing collected packages: numpy, zipp, importlib-resources, six, typing-extensions, ply, thriftpy2, happybase
Successfully installed happybase-1.3.0 importlib-resources-5.12.0 numpy-1.21.6 ply-3.11 six-1.17.0 thriftpy2-0.6.0 typing-extensions-4.7.1 zipp-3.15.0
bash-5.0#
```

HBase Thrift server running:

```
bash-5.0# nohup hbase thrift start &
[1] 6225
bash-5.0# nohup: ignoring input and appending output to 'nohup.out'
jps
656 SecondaryNameNode
6225 ThriftServer
514 NameNode
756 ResourceManager
2870 Master
2938 HMaster
2874 HistoryServer
1083 RunJar
6348 Jps
1084 RunJar
1054 JobHistoryServer
bash-5.0#
```

The HappyBase and Numpy packages were installed to support integration between Spark and HBase. Numpy provides numerical functionality for data processing tasks, while HappyBase allows Python applications to communicate with HBase through the Thrift interface. The HBase Thrift server was started to enable external connections from Python based Spark jobs. These steps were necessary to ensure that processed data and model metrics can be written from Spark into HBase as part of the project pipeline.

Objective 4: HBase Table Creation

```
hbase(main):001:0> create 'water_metrics', 'cf'
Created table water_metrics
Took 1.1801 seconds
=> Hbase::Table - water_metrics
hbase(main):002:0> scan 'water_metrics'
ROW                                COLUMN+CELL
0 row(s)
Took 0.1591 seconds
hbase(main):003:0> █
```

In HBase, row keys and columns are created when data is inserted, not when the table is created. The row key for water_metrics table will uniquely identify each Spark model evaluation run, allowing multiple model results to be stored without overwriting previous entries. A single column family named cf was used to group related evaluation metrics such as accuracy, precision, recall, and F1 score. This design keeps the schema simple while allowing flexible addition of new metric columns in the future.

Objective 5: PySpark ML Code

GitHub Link: <https://raw.githubusercontent.com/josierra21/bigData-DSC650/refs/heads/main/PySparkML.py>

Training output:

```
43033 [Thread-5] INFO org.apache.spark.scheduler.DAGScheduler - Job 5 finished: collectAsMap at MulticlassMetrics.scala:61, took 0.691760 s
AUC: 0.0
Accuracy: 1.0
43171 [Thread-5] INFO org.apache.spark.SparkContext - Starting job: foreachPartition at /usr/program/PySparkML.py:88
43174 [dag-scheduler-event-loop] INFO org.apache.spark.scheduler.DAGScheduler - Got job 6 (foreachPartition at /usr/program/PySparkML.py:88) with 2 output partitions
43174 [dag-scheduler-event-loop] INFO org.apache.spark.scheduler.DAGScheduler - Final stage: ResultStage 14 (foreachPartition at /usr/program/PySparkML.py:88)
43174 [dag-scheduler-event-loop] INFO org.apache.spark.scheduler.DAGScheduler - Parents of final stage: List()
43174 [dag-scheduler-event-loop] INFO org.apache.spark.scheduler.DAGScheduler - Missing parents: List()
43176 [dag-scheduler-event-loop] INFO org.apache.spark.scheduler.DAGScheduler - Submitting ResultStage 14 (PythonRDD[72] at foreachPartition at /usr/program/PySparkML.py:88), which has no missing
parents
```

For this project, Logistic Regression was selected because the target variable, Potability, is binary, making this a classification problem. The numeric water quality features were assembled into a feature vector using Spark's VectorAssembler, and the model was trained using a train-test split. The model was evaluated using Accuracy and AUC. The results showed Accuracy = 1.0 and AUC = 0.0, which suggests the dataset may be imbalanced or dominated by one class (dataset was cut due to VM limitations). The successful execution shows that the Spark ML pipeline trained correctly and generated evaluation metrics as expected.

Objective 6: Spark Submit & Output

Spark submit:

```
bash-5.0# spark-submit \
> --master yarn \
> --deploy-mode client \
> --name WaterML_to_HBase \
> PySparkML.py
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/usr/program/spark/jars/slf4j-log4j12-1.7.30.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/usr/program/hadoop/share/hadoop/common/lib/slf4j-log4j12-1.7.25.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.slf4j.impl.Log4jLoggerFactory]
0 [main] WARN org.apache.hadoop.util.NativeCodeLoader - Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
1435 [Thread-5] INFO org.apache.spark.SparkContext - Running Spark version 3.0.0
1511 [Thread-5] INFO org.apache.spark.resource.ResourceUtils - =====
1513 [Thread-5] INFO org.apache.spark.resource.ResourceUtils - Resources for spark.driver:

1513 [Thread-5] INFO org.apache.spark.resource.ResourceUtils - =====
1515 [Thread-5] INFO org.apache.spark.SparkContext - Submitted application: Water Potability MLlib Prediction
1594 [Thread-5] INFO org.apache.spark.SecurityManager - Changing view acls to: root
1594 [Thread-5] INFO org.apache.spark.SecurityManager - Changing modify acls to: root
1594 [Thread-5] INFO org.apache.spark.SecurityManager - Changing view acls groups to:
1595 [Thread-5] INFO org.apache.spark.SecurityManager - Changing modify acls groups to:
1595 [Thread-5] INFO org.apache.spark.SecurityManager - SecurityManager: authentication disabled; ui acls disabled; users with view permissions: Set(root); groups with
permissions: Set(root); groups with modify permissions: Set()
```

Spark job logs:

```
bash-5.0# yarn application -list -appStates FINISHED
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/usr/program/hadoop/share/hadoop/common/lib/slf4j-log4j12-1.7.25.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/usr/program/tez/lib/slf4j-log4j12-1.7.10.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/usr/program/hive/lib/log4j-slf4j-impl-2.10.0.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.slf4j.impl.Log4jLoggerFactory]
2026-03-03 02:14:17,197 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
2026-03-03 02:14:17,308 INFO client.RMProxy: Connecting to ResourceManager at master/172.28.1.1:8032
Total number of applications (application-types: [], states: [FINISHED] and tags: []) : 7


| Application-Id                 | Application-Name                          | Application-Type | User | Queue | root      | root:root | State    | Final-State | Progress  | 100% | Tracking-URL | N/A |
|--------------------------------|-------------------------------------------|------------------|------|-------|-----------|-----------|----------|-------------|-----------|------|--------------|-----|
| application_1772496128396_0007 | Water Potability MLlib Prediction         | SPARK            | root | root  | root:root | root:root | FINISHED | FINISHED    | SUCCEEDED | 100% |              | N/A |
| application_1772496128396_0008 | Water Potability MLlib Prediction         | SPARK            | root | root  | root:root | root:root | FINISHED | FINISHED    | SUCCEEDED | 100% |              | N/A |
| application_1772496128396_0002 | HIVE-lacdec07-c5f6-4923-ab13-3637a52541c  | TEZ              | root | root  | root:root | root:root | FINISHED | FINISHED    | SUCCEEDED | 100% |              | N/A |
| N/A                            |                                           |                  |      |       |           |           |          |             |           |      |              |     |
| application_1772496128396_0005 | HIVE-bfc7de81-6919-44ac-b46d-d806d5040b33 | TEZ              | root | root  | root:root | root:root | FINISHED | FINISHED    | SUCCEEDED | 100% |              | N/A |
| N/A                            |                                           |                  |      |       |           |           |          |             |           |      |              |     |
| application_1772496128396_0006 | Water Potability MLlib Prediction         | SPARK            | root | root  | root:root | root:root | FINISHED | FINISHED    | SUCCEEDED | 100% |              | N/A |
| application_1772496128396_0003 | HIVE-d105883-2783-472c-93b1-42c8a32ac35   | TEZ              | root | root  | root:root | root:root | FINISHED | FINISHED    | SUCCEEDED | 100% |              | N/A |
| N/A                            |                                           |                  |      |       |           |           |          |             |           |      |              |     |
| application_1772496128396_0004 | HIVE-9b5ab5e0-1752-43ce-b7c1-d4dc79dd6ddb | TEZ              | root | root  | root:root | root:root | FINISHED | FINISHED    | SUCCEEDED | 100% |              | N/A |
| N/A                            |                                           |                  |      |       |           |           |          |             |           |      |              |     |


```

Job application_1772496128396_0007 (Water Potability MLlib Prediction):

```
bash-5.0# yarn logs -applicationId application_1772496128396_0007
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/usr/program/hadoop/share/hadoop/common/lib/slf4j-log4j12-1.7.25.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/usr/program/tez/lib/slf4j-log4j12-1.7.10.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/usr/program/hive/lib/log4j-slf4j-impl-2.10.0.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.slf4j.impl.Log4jLoggerFactory]
2026-03-03 02:17:41,493 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
2026-03-03 02:17:41,583 INFO client.RMProxy: Connecting to ResourceManager at master/172.28.1.1:8032
Container: container_1772496128396_0007_01_000002 on worker1_38321
LogAggregationType: AGGREGATED
=====
LogType:directory.info
LogLastModifiedTime:Tue Mar 03 01:52:35 +0000 2026
LogLength:37158
LogContents:
ls -l:
total 96
```

```
Container: container_1772496128396_0007_01_000002 on worker1_38321
LogAggregationType: AGGREGATED
=====
LogType:prelaunch.out
LogLastModifiedTime:Tue Mar 03 01:52:35 +0000 2026
LogLength:100
LogContents:
Setting up env variables
Setting up job resources
Copying debugging information
launching container

End of LogType:prelaunch.out
=====
Container: container_1772496128396_0007_01_000002 on worker1_38321
LogAggregationType: AGGREGATED
=====
LogType:stderr
LogLastModifiedTime:Tue Mar 03 01:52:35 +0000 2026
LogLength:492
LogContents:
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/tmp/hadoop-root/nm-local-dir/filecache/252/slf4j-log4j12-1.7.30.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/usr/program/hadoop/share/hadoop/common/lib/slf4j-log4j12-1.7.25.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.slf4j.impl.Log4jLoggerFactory]

End of LogType:stderr
```

Objective 7: HBase Scan Verification

```
hbase(main):001:0> scan 'water_metrics'
ROW                                COLUMN+CELL
run_20260303_095230                column=cf:accuracy, timestamp=2026-03-03T01:52:33.984, value=1.0
run_20260303_095230                column=cf:auc, timestamp=2026-03-03T01:52:33.986, value=0.0
run_20260303_100812                column=cf:accuracy, timestamp=2026-03-03T02:08:13.787, value=1.0
run_20260303_100812                column=cf:auc, timestamp=2026-03-03T02:08:13.785, value=0.0
2 row(s)
Took 0.6216 seconds
hbase(main):002:0> █
```

The HBase scan output confirmed that Spark successfully wrote evaluation metrics into the water_metrics table. Each row key represents a unique model run, and the column family cf stores the model performance metrics, including accuracy and AUC. These values verify that the pipeline executed successfully from Hive data loading to Spark model training and finally to long-term storage in HBase.

Issues Encountered:

During the formation of the pipeline, some technical issues were encountered while configuring and integrating the different components of the Hadoop ecosystem. Some challenges involved managing Docker containers and ensuring that services such as Hive, HBase, and Spark were running correctly within the VM environment. There were also issues related to missing Python dependencies when running the Spark machine learning script, which required installing additional packages such as HappyBase and NumPy on the worker nodes so that the Spark executors could access them. Another challenge involved ensuring that data was properly written to HDFS and correctly loaded into Hive tables before it could be processed by Spark. These issues required troubleshooting container configurations, verifying service status, and re-running certain pipeline steps.

Conclusion:

This project demonstrated how a distributed data pipeline can be built using several components of the Hadoop ecosystem working together. Data was ingested using Apache NiFi, stored in HDFS, queried through Hive, processed with Spark MLlib, and the resulting model metrics were stored in HBase. Running these tools within a Google Cloud VM environment helped illustrate how large-scale data systems are typically deployed and managed. The pipeline showed how data can move through multiple stages of processing while remaining scalable and organized across distributed services. Overall, the project provided practical experience with designing and operating a complete big data workflow and highlighted how different technologies integrate to support modern data processing pipelines.

Code

Code File (NiFi Template) : https://raw.githubusercontent.com/bellevue-university/dsc650-updated/refs/heads/main/week11-12/assignment/examples/Final_Project.json

Objective 2 Query :

```
SELECT COUNT(*) AS acidic_water  
FROM water_quality  
WHERE ph < 7;
```

Code File: PySparkML.py

<https://raw.githubusercontent.com/josierra21/bigData-DSC650/refs/heads/main/PySparkML.py>

```
from pyspark.sql import SparkSession  
from pyspark.ml.feature import VectorAssembler  
from pyspark.ml.classification import LogisticRegression  
from pyspark.ml.evaluation import BinaryClassificationEvaluator,  
MulticlassClassificationEvaluator  
import happybase  
from datetime import datetime
```

Step 1: Create a Spark session (Hive support enabled)

```
spark = SparkSession.builder \  
    .appName("Water Potability MLlib Prediction") \  
    .enableHiveSupport() \  
    .getOrCreate()
```

Step 2: Load data from the Hive table into a Spark DataFrame

```
water_df = spark.sql("""  
    SELECT  
        ph, Hardness, Solids, Chloramines, Sulfate, Conductivity,  
        Organic_carbon, Trihalomethanes, Turbidity,  
        Potability  
    FROM water_quality  
""")
```

Step 3: Handle null values

```
water_df = water_df.na.drop()
```

```
# Step 4: Assemble features into a single vector column
```

```
feature_cols = [  
    "ph", "Hardness", "Solids", "Chloramines", "Sulfate", "Conductivity",  
    "Organic_carbon", "Trihalomethanes", "Turbidity"  
]
```

```
assembler = VectorAssembler(  
    inputCols=feature_cols,  
    outputCol="features",  
    handleInvalid="skip"  
)
```

```
assembled_df = assembler.transform(water_df) \  
    .select("features", water_df["Potability"].alias("label"))
```

```
# Step 5: Split into training and testing sets
```

```
train_data, test_data = assembled_df.randomSplit([0.7, 0.3], seed=42)
```

```
# Step 6: Train Logistic Regression model (binary classification)
```

```
lr = LogisticRegression(featuresCol="features", labelCol="label", maxIter=50)  
model = lr.fit(train_data)
```

```
# Step 7: Make predictions
```

```
predictions = model.transform(test_data)
```

```
# Step 8: Evaluate model (AUC + Accuracy)
```

```
auc_evaluator = BinaryClassificationEvaluator(  
    labelCol="label",  
    rawPredictionCol="rawPrediction",  
    metricName="areaUnderROC"  
)
```

```
auc = auc_evaluator.evaluate(predictions)
```

```
acc_evaluator = MulticlassClassificationEvaluator(  
    labelCol="label",  
    predictionCol="prediction",
```

```
    metricName="accuracy"
)
accuracy = acc_evaluator.evaluate(predictions)

print(f"AUC: {auc}")
print(f"Accuracy: {accuracy}")

# Write metrics to HBase with happybase
# Table: water_metrics, Column Family: cf
# Row key: unique run id so each run is stored separately
run_id = "run_" + datetime.now().strftime("%Y%m%d_%H%M%S")

data = [
    (run_id, "cf:auc", str(auc)),
    (run_id, "cf:accuracy", str(accuracy))
]

def write_to_hbase_partition(partition):
    connection = happybase.Connection("master")
    connection.open()
    table = connection.table("water_metrics")
    for row_key, column, value in partition:
        table.put(row_key, {column: value})
    connection.close()

rdd = spark.sparkContext.parallelize(data)
rdd.foreachPartition(write_to_hbase_partition)

# Stop Spark session
spark.stop()
```